

SemantOS: Safe and Explainable Kernel Tuning via Semantic Reasoning and Guardrailed LLMs

Anonymous submission

Abstract

SemantOS investigates a kernel-tuning control loop combining telemetry, a typed interaction graph, retrieval-augmented language-model recommendations, and independent enforcement. This revision distinguishes the implemented prototype from the intended deployment system. The artifact provides a dry-run bundle gate, provenance-carrying recommendations, and a class-conditional rank threshold with explicit assumptions. We replace unsupported performance and drift-recovery claims with a corrected expected-cost identity, an auditable measurement procedure, and a matched-information evaluation protocol. We also report newly collected baseline timings for three local microbenchmarks, with raw operation logs and repeated-run uncertainty; these measurements do not establish tuning gains. A further 9,600-event experiment measures actual per-process timer-slack and affinity interventions; a training-selected configuration matches the default control, with no selected-policy gain. Whether language-model reasoning improves on a conventional optimizer supplied with the same graph, traces, and budget remains an open empirical question. A separate author-reported hardware study describes a 14% aggregate P95 reduction; raw logs and the aggregation rule were not supplied, so this estimate is not independently reproduced. No production-safety guarantee or human-study outcome is claimed.

Keywords. Kernel tuning; knowledge representation; selective prediction; provenance; experimental methodology.

Introduction

Kernel tuning requires choosing configurations whose effects depend on workload, hardware, and the current operating state. A setting that improves one metric can worsen another. SemantOS explores whether an explicit representation of empirical interactions, combined with language-model proposals and an independent runtime, can make this process inspectable. The central research question is not whether a model can describe a configuration, but whether its decisions add value beyond the information already present in the knowledge base (KB).

The proposed architecture separates three responsibilities: evidence acquisition, candidate generation, and enforcement.

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

A typed graph and retrieved traces expose the available evidence; a reasoner proposes a multi-knob bundle; a runtime checks the complete bundle and decides whether to veto it or enter a staged rollout. This separation makes the reasoner’s contribution testable against graph-aware conventional optimizers.

Evidence status. The earlier artifact generated numerical outputs from constants matching manuscript tables, including Gaussian perturbations of specified effect sizes. Such a program checks a simulation’s internal consistency, not an independently measured system. We have quarantined those artifacts and removed their quantitative claims from the current argument. The same applies to unsupported descriptions of a trained checkpoint, a 30-day deployment, and a 32-participant study. Their removal does not establish that no external records exist; it states that they are not evidence available in this artifact.

Contributions and scope. This revision provides (i) an explicit bundle-level control contract and a dry-run implementation, (ii) a corrected cost identity and a rank-gate result whose conditioning and assumptions are stated precisely, and (iii) raw-data measurement and evaluation procedures that separate independent outcomes from enforcement decisions. The new local measurements demonstrate the operation of the measurement pipeline only. Deployment efficacy and the incremental value of the LLM require the experiments in Section .

Background and Related Work

Kernel interfaces. Linux documents controls such as `vm.swappiness`, `vm.dirty_ratio`, and `vm.dirty_background_ratio` (Linux kernel contributors 2024). Their availability and semantics must be checked on the actual kernel. A feature name is not necessarily a `sysctl`: the scheduler’s `WAKE_AFFINE` feature in Linux 6.4 does not establish the existence of a `sched_wake_affinity` `sysctl` (Linux kernel contributors 2023). The current example policy uses a small allowlist of documented memory controls and makes no scheduler-pair performance claim.

Optimization with shared information. Black-box optimization need not assume independent parameters. A sur-

rogate can model interactions, and a graph-aware optimizer can receive exactly the evidence given to an LLM. Therefore a comparison that grants offline pairwise-search results only to SemantOS cannot isolate language-model reasoning. Our evaluation protocol includes both cold-start and shared-KB baselines, and accounts separately for offline and online costs. We remove the earlier categorical superiority table, including its inconsistent classification of OS-R1, rather than infer unverified properties of other systems.

Risk control. Conformal Risk Control (Angelopoulos et al. 2024) treats expected monotone losses, providing a more appropriate reference than an informal conversion of coverage into conditional acceptance risk. Non-Exchangeable Conformal Risk Control (Farinhas et al. 2024) addresses weighted, non-exchangeable data; it does not imply that an arbitrary sliding window restores a fixed risk level. Our rank argument below is a restricted class-conditional result, not a replacement for these frameworks or a novel drift theorem.

Explanation faithfulness. Provenance lists what context was supplied, not necessarily what influenced a recommendation. ContextCite (Cohen-Wang et al. 2024) studies attribution of generated output to context. We adopt context-removal interventions as a planned test of whether cited edges and traces influence the proposed bundle. Operator preference alone cannot establish faithfulness.

Model provenance. Meta’s Llama 3.1 release comprises 8B, 70B and 405B variants (Meta 2024). The prior 13B designation was erroneous. The artifact now requires explicit configuration of an installed model and describes an 8B configuration only as a proposed example, not as the checkpoint used in an experiment.

Motivation and Testable Questions

SemantOS is motivated by three hypotheses rather than by the earlier unverified 1,000-pair study. First, a configuration’s utility may depend on workload and hardware. Second, joint knob effects may be non-additive. Third, performance, failure probability, and intervention cost may conflict. Each hypothesis requires measured interventions with explicit baselines.

For a loss L , define an interaction contrast from four independently measured configurations:

$$I_{ab} = L(a, b) - L(a, b_0) - L(a_0, b) + L(a_0, b_0).$$

For lower-is-better loss, negative I_{ab} indicates a beneficial non-additive interaction relative to the chosen baseline. A bootstrap interval must resample independent runs, not individual requests from a single run. The sign can change with workload, value range, or kernel version. An edge therefore needs its scope, configuration values, measurement IDs and uncertainty interval. Such an edge describes an empirical interaction; it is not an unconditional causal coupling.

A **DEPENDS-ON** edge requires a validated admissibility rule or measured range conditioning. It cannot be inferred solely from the sign of I_{ab} . The current seed graph is empty: unverified scheduler edges are not silently renamed to other controls. The principal question is whether a reasoner can

use independently acquired evidence more effectively than an optimizer with the same inputs, after charging the cost of acquiring that evidence.

SemantOS Design and Implemented Scope

Control Contract

The intended architecture contains telemetry, a graph and trace store, a reasoner, a runtime and an operator console. The current REST prototype connects these components, but its runtime is explicitly a *dry run*: it stages an in-memory bundle and never writes a kernel setting, routes traffic or demonstrates a kernel rollback.

Evidence and score. The reasoner retrieves typed neighbors and trace text, then requests $k = 3$ model samples. For a knob-value pair proposed in v samples, its current heuristic score is $u = 0.7(1 - v/k) + 0.3(1 - w)$, clipped to $[0.02, 1]$, where w is the strongest retrieved synergy weight (zero when absent). This is neither predictive entropy nor a calibrated probability. Backend failure must be recorded; repeated copies of a fallback do not constitute independent model votes. The implementation abstains when no configured model produces recommendations. Supplied evidence is recorded as provenance, not certified as a faithful explanation.

Bundle contract. A request contains a nonempty bundle identifier and a list of recommendations with `id`, `knob`, `proposed`, `uncertainty`, and optional `rationale` and `provenance`. Each request submits one bundle. Duplicate knobs, nonfinite scores, unsupported controls, and example-policy bound violations reject the entire request. Dirty-ratio changes must supply both values and satisfy the example policy that the background ratio is less than the foreground ratio. This is a declared policy, not proof that a configuration improves any workload. If any score satisfies $u \geq \tau$, all members are vetoed; otherwise all are staged together. A concurrent bundle receives a conflict response. Dry-run results return `applied=[]` and list simulated members separately.

Deployment requirements. A real actuator must snapshot all old values, validate the full target configuration and available host interfaces, serialize updates, verify readback, and restore every old value on failure. Multiple `sysctl` writes are not a kernel transaction: partial exposure must be measured and cannot be described as atomic hardware state. Canary shares require separate hosts or an appropriate isolation mechanism; changing a host-global `sysctl` cannot isolate a fraction of requests on that host. Missing, stale or malformed end-to-end telemetry must stop promotion. These capabilities are requirements, not features demonstrated by the dry-run state machine.

One illustrative decision trace. Consider supplied context with example values 5 and 15 for the two dirty ratios, and retrieved evidence IDs e_1, t_1 . Suppose all three model samples agree on this pair and $w = 0$, giving $u = 0.30$. If an independently obtained threshold is $\tau = 0.25$, the whole bundle is vetoed; if $\tau = 0.35$, it can enter the dry-run canary after joint validation. The audit record includes the values,

Algorithm 1 Implemented dry-run bundle path

Require: Validated bundle B , fixed score function, calibration threshold τ

- 1: Validate every member and joint constraints before changing state
 - 2: **if** another bundle is active **then**
 - 3: Return conflict
 - 4: **end if**
 - 5: **if** $\max_{a \in B} u(a) \geq \tau$ **then**
 - 6: Record veto for all members; return
 - 7: **end if**
 - 8: Stage the complete bundle in memory at 5%
 - 9: **for** requested stage in $\{25, 50, 100\}\%$ **do**
 - 10: Advance the simulated bundle; record the transition
 - 11: **end for**
 - 12: On cancellation, clear the complete simulated bundle
 - 13: Never infer an unsafe/safe training label from simulated completion
-

scores, threshold and decision. These numbers illustrate the contract and are not a measured recommendation or calibrated operating point. The initial artifact threshold is zero until calibration data is explicitly supplied.

Unimplemented scheduling claims. The earlier 94% fast-path, 6% asynchronous LLM path, 10ms latency and resource-footprint claims are withdrawn. The current reasoner awaits model samples. A future nonblocking deployment path should separate synchronous policy enforcement from asynchronous proposals, record the objective weights and policy version per decision, and refresh graph and calibration snapshots with versioned evidence. No *delay* state or immutable audit storage is claimed: logs are append-only files within a process, without tamper resistance.

Corrected Probability Statements

Let $Y = 1$ denote an independently defined unsafe outcome and $A = 1$ acceptance. An outcome label is not defined by whether a rollback happened. For example, a benchmark can predeclare a request deadline, count observed failures or deadline exceedances, and define an action-level breach from these raw observations in a fixed window. Rejected actions have unknown counterfactual outcomes unless evaluated in an isolated experiment; they must not automatically be labeled safe or unsafe.

Theorem 1 (Cost identity). *For costs $c_{FN}, c_{FP}, \lambda \geq 0$, define $C = c_{FN}\mathbf{1}\{Y = 1, A = 1\} + c_{FP}\mathbf{1}\{Y = 0, A = 0\} + \lambda D$, where $D \geq 0$ is measured exposure or SLO debt. Write $\pi = \Pr(Y = 1)$, $q = \Pr(A = 1 \mid Y = 1)$ and $r = \Pr(A = 0 \mid Y = 0)$. Then*

$$\mathbb{E}C = c_{FN}\pi q + c_{FP}(1 - \pi)r + \lambda \mathbb{E}D.$$

For a zero-probability class its associated product is zero.

Proof. Expand each indicator expectation as its joint probability and use the definition of conditional probability. Linearity requires no independence. $\square$

This is an accounting identity, not a claim of novelty or a safety bound. A bound $q \leq \alpha$ controls misses among unsafe actions. A different bound $\Pr(Y = 1 \mid A = 1) \leq \alpha$ instead gives $\Pr(Y = 1, A = 1) \leq \alpha \Pr(A = 1)$, not $\pi\alpha$. Likewise r is a veto rate among safe actions, not the proportion of vetoed actions that are safe. We report all denominators separately.

Proposition 1 (Restricted rank gate). *Fix a score function before calibration. Suppose n unsafe calibration scores and the next unsafe test score are jointly exchangeable. Set $j = \lfloor \alpha(n + 1) \rfloor$ for $0 < \alpha < 1$. If $j = 0$, veto all; otherwise let τ be the j th smallest unsafe calibration score, and accept only when $u < \tau$. Then the unsafe-class acceptance probability, marginal over calibration and test scores, is at most $j/(n + 1) \leq \alpha$.*

Proof. With distinct scores, the test score has a uniform rank among $n + 1$ exchangeable scores. It lies strictly below the j th calibration order statistic only in the first j ranks. Ties with strict acceptance can only reduce this probability; $j = 0$ accepts nothing. $\square$

For bundle-level calibration, u denotes the maximum member score and Y the independently observed bundle outcome. Per-knob labels alone do not establish a bundle-level guarantee.

Selecting a smaller threshold by a cost rule cannot enlarge the acceptance event. Selecting a larger threshold can invalidate the bound; therefore the implementation uses a minimum, not the former maximum, of the rank threshold and optional cost-selected cap. No unsafe calibration examples implies veto-all. Calibration-window diagnostics are not test-set estimates.

Drift and selective observation. Sliding windows and a drift detector do not guarantee exchangeability or a deterministic recovery time. The earlier recovery inequality and 30-day confirmation are withdrawn. Under an explicitly justified total-variation bound on the *joint calibration–test law*, the probability of the same event changes by at most that bound; a marginal score-distance statement alone does not establish it. This revision neither estimates that distance nor claims recovery. Online labels selected by acceptance introduce additional selection bias. The runtime holds its threshold at zero on detected change until an explicit refresh; refresh remains an exploratory operation, not proof of safety.

Measurements and Evaluation Protocol

New Local Measurements

We executed three bounded microbenchmarks on the available host: an Intel Core i5-3570 with four logical CPUs, approximately 31.2 GiB RAM, and Linux 6.17.0-23-generic. These are newly collected measurements, not the earlier six-workload, three-server evaluation. No kernel knob was modified and no LLM or controller was compared.

Each workload had 10 separate invocations of 100 measured operations, preceded by 10 warm-up operations. Workload–run order was shuffled with seed 4088. SQLite inserts a 4 KiB value and commits each transaction in WAL mode with synchronous=FULL. The file test overwrites a

Table 1: Measured local baseline. Mean of run medians, and mean of run P95 values with 95% Student-*t* CI half-width across 10 runs. Last column is the percentage of errors or operations exceeding a predeclared 50 ms deadline. No tuning gain is implied.

Workload	Median ms	P95 ms	Events %
file_fsync	1.903	2.035 ± 0.043	0.00
sqlite_commit	1.870	2.187 ± 0.273	0.00
zlib_compress	1.474	2.038 ± 0.356	0.00

64 KiB region and calls fsync. The compression test compresses a fixed seeded 64 KiB payload at zlib level 6 and verifies decompression. Latency is wall-clock operation duration from a monotonic nanosecond clock; the compression measurement includes verification. The raw log retains every operation, run ID, deadline and error flag, with a SHA-256 hash and software/environment manifest.

The anomaly denominator is all measured operations, including failures; an operation counts once if either it fails or exceeds the deadline. This label is independent of veto and rollback. Zero events in these short runs is not evidence of zero population risk. Run-level intervals describe variability on this shared host and do not account for uncontrolled interference, warm-cache bias or correlated system state. The closed-loop load does not characterize queueing under an externally fixed arrival rate. We do not pool heterogeneous workload latencies or infer kernel tuning effects from these baseline timings.

Measured Per-Process Kernel Interventions

We additionally executed real Linux timer-slack and CPU-affinity interventions in fresh owned child processes, without modifying host-wide sysctls. The two factors were timer slack (50 μ s or 1 ms, set by PR_SET_TIMERSLACK) and affinity (all four available CPUs or CPU 0). Every child checked kernel readback before recording events. This isolated measurement driver is separate from the dry-run SemantOS actuator.

For each periodic wake-up interval (2 or 5 ms), five training blocks and ten subsequent test blocks evaluated all four configurations. Within each block, the eight interval/configuration cases were randomly ordered using seed 4088. Each fresh process performed ten warm-up and 80 measured wake-ups against absolute monotonic-clock targets. The plan was written before execution; the configuration minimizing mean training-run P95 lateness was frozen before test execution. This is temporal holdout within one host and workload family, not the independent workload/hardware split required for claims of generalization. The complete 120-process experiment retained 9,600 event timestamps, readbacks, frozen selections and hashes. No external competing load was introduced.

The training selector chose the explicit control (50 μ s slack, all CPUs) at both intervals: there is no selected-policy improvement over that control. Its test P95 was 89.5 ± 9.2 and $94.2 \pm 6.9 \mu$ s at 2 and 5 ms, respectively (95% Student-*t*

Table 2: Held-out kernel intervention measurements. P95 is the mean of ten run-level P95 wake-up latenesses. Misses count lateness exceeding the predeclared 200 μ s deadline. CPU affinity is all four CPUs or one CPU.

Period	Slack/CPU	P95 (μ s)	Misses
2	50us/all	89.5	2/800
2	50us/one	109.4	18/800
2	1ms/all	1016.8	800/800
2	1ms/one	968.9	771/800
5	50us/all	94.2	2/800
5	50us/one	133.8	22/800
5	1ms/all	1022.6	663/800
5	1ms/one	1073.7	680/800

half-width across ten runs). Raising slack to 1 ms with all CPUs increased P95 by 927.3 ± 16.4 and $928.4 \pm 10.0 \mu$ s in paired blocks. Affinity-only differences were 19.9 ± 36.7 and $39.6 \pm 62.6 \mu$ s; these intervals include zero. These exploratory intervals are unadjusted for multiple comparisons and shared-host temporal dependence. The slack effect is consistent with timer coalescing; no power measurement, graph discovery, LLM advantage, staged rollback or six-workload result is inferred. A miss is defined by observed wake-up time, independently of any safety gate.

Required Controlled Comparisons

Matched information and budgets. To test the LLM’s incremental value, compare (1) the unchanged kernel baseline, (2) a frozen expert profile, (3) a conventional optimizer without the KB, (4) the same optimizer with identical graph and retrieved traces, (5) a deterministic graph policy, and (6) SemantOS. Fix the allowed knob values, workload arrivals, hardware allocation, trial and wall-clock budgets, safety policy and objective weights. Record model inference, graph construction, retrieval, calibration and human setup costs separately. Report both online and total costs; a cold-start comparison alone cannot isolate semantic reasoning.

Ablations remove graph context, retrieval, or the LLM while retaining the other inputs and the runtime. A no-runtime comparison belongs only in an isolated testbed. The KB may carry all useful information; the shared-KB optimizer is the direct test of that alternative. Distinct random-search and optimizer implementations must not be merged into a generic BO/RL result.

Leakage prevention. Before graph induction, define immutable groups by workload family, hardware class and acquisition session. Maintain disjoint train, retrieval, calibration and test roles; no held-out group may contribute to LLM training, pairwise sweeps, retrieval traces or threshold selection. Freeze hashes of graph snapshots, source measurements, prompts and calibrators before opening test results. Temporal reuse must be forward-only. The artifact includes a manifest validator to reject group overlap and unknown lineage. The old synthetic split, which held out only selected labels, did not establish independence for all six workloads and all three server classes.

Outcome definitions and safety counts. For each proposed bundle record proposed, malformed, constraint-rejected, score-vetoed, accepted, staged, completed and rolled-back counts, together with exposure duration and affected requests. Count bundles and individual knobs separately. For independently labeled actions report TP, FP, FN and TN for the *gate*; separately report rollout rollback counts and causes. Give both `#unsafe accepted/#unsafe labeled` and `#unsafe accepted/#accepted labeled`. Unknown labels remain unknown. Rollback precision is undefined without independently labeled executed actions; veto precision is not its substitute.

A lower threshold accepts a subset of the same fixed candidate list, but the unsafe fraction among survivors need not be monotone. Report absolute unsafe counts, accepted counts, request-level deadline misses and exposure under each threshold on the same frozen candidates. The earlier precision-derived anomaly formula is not an acceptable measurement.

Aggregation and trade-offs. Use paired seeds and runs and report absolute per-workload and per-host metrics before percentage reductions against the explicitly named unchanged-kernel baseline. Aggregate normalized ratios, not raw milliseconds across heterogeneous tasks. Bootstrap independent runs or workload-host groups as appropriate; state the unit of resampling. A Pareto plot requires multiple measured operating points per method across a common budget or objective-weight sweep. Connect only genuinely nondominated measured points and report uncertainty; a dashed line through one point per method is not evidence of a frontier.

Drift and faithfulness. A drift experiment must retain the score function version, α , window size, number of unsafe calibration examples, change schedule, threshold trajectory and held-out outcomes at each decision. Evaluate on subsequent data, not on the window used to select the threshold. Report observed misses, denominators and uncertainty; any claimed recovery deadline needs its own assumptions and evidence. For explanation faithfulness, delete cited edges/traces and matched uncited controls while fixing model seeds and objective weights, then measure bundle changes and attribution errors. Record the complete original and perturbed prompts. No human study is needed to begin this intervention test, and no user-study outcome is asserted here.

Additional author-reported measurements. Appendix records a separately supplied hardware experiment report, which the authors confirm contains actual measurements. Its reported aggregate P95 decreases from 20.0 to 17.2 ms (14.0%), but workload-level aggregation and run-level uncertainty remain unresolved. These externally supplied summaries are distinguished from the locally executed measurements above; no raw logs were supplied for independent recomputation.

Discussion and Limitations

The central idea remains the separation of structured evidence, proposals, and enforcement. The current artifact

demonstrates a dry-run decision contract and auditable baseline and per-process intervention measurements, not end-to-end kernel optimization. It lacks a deployed bundle actuator, traffic isolation, measured graph induction, a verified trained checkpoint, and controlled end-to-end comparative results. Prompting a model to obey graph edges does not enforce those edges; independent validation is required. The current runtime enforces only its explicit example constraints.

The available telemetry prototype contains scheduler and block-I/O demonstration proxies, including timestamp-derived buckets; those are not end-to-end request latencies. They must not calibrate an empirical SLO claim. Actual workload drivers must record completion, failure and deadline data. The new local benchmarks do so for their narrow tasks but do not substitute for web serving, TPC-C, Kafka/Spark, GPU inference or audio pipelines.

The rank gate’s assumptions can fail under drift, selective labeling, or a score model updated using calibration data. A canary can suffer harm before detection, and rollback cannot undo every side effect. No no-worse-than-default, catastrophic-regression prevention, Pareto superiority or universal safety claim follows from the prototype. These limitations define concrete measurements and engineering prerequisites for the next evaluation.

Ethical and Responsible Deployment

The new measurements run local software workloads and collect no participant data. No human study or IRB approval is claimed in this revision. The current runtime performs no kernel writes. Future interventions require an isolated testbed, documented rollback limits and appropriate authorization. Human studies, if pursued, require prospective institutional review as applicable and informed consent; these cannot be inferred from software tests.

An AI coding assistant assisted this revision’s code, prose and analysis. Measurement provenance, source hashes and explicit evidence limits are retained for author verification. Generated values are not substituted for unavailable observations.

Conclusion

SemantOS is a research prototype for inspectable, knowledge-grounded kernel-tuning proposals and independent bundle gating. This revision corrects the risk statements, separates simulation from measurement, and reports auditable local baselines and actual per-process kernel interventions. Whether the LLM improves decisions over an equally informed optimizer, and whether staged deployment reduces independently measured harm, remain questions for the specified controlled experiments.

A separate author-reported hardware evaluation describes lower tail latency for Graph + LLM and Full SemantOS. Its aggregate estimates are included with their provenance, while unresolved aggregation, missing run-level uncertainty and absent raw logs limit the strength of comparative conclusions.

References

- Angelopoulos, A. N.; Bates, S.; Fisch, A.; Lei, L.; and Schuster, T. 2024. Conformal Risk Control. In *International Conference on Learning Representations*. .
- Cohen-Wang, B.; Shah, H.; Georgiev, K.; and Mađry, A. 2024. ContextCite: Attributing Model Generation to Context. In *Advances in Neural Information Processing Systems*, volume 37. .
- Farinhas, A.; Zerva, C.; Ulmer, D.; and Martins, A. F. T. 2024. Non-Exchangeable Conformal Risk Control. In *International Conference on Learning Representations*. .
- Linux kernel contributors. 2023. Linux v6.4 scheduler features. .
- Linux kernel contributors. 2024. Documentation for /proc/sys/vm/. .
- Meta. 2024. Introducing Llama 3.1: Our most capable models to date. .

Table 3: Author-reported measured aggregate values. Relative changes are computed from the supplied aggregates; the aggregation rule and run-level uncertainty are unavailable. Anomaly reduction is 1.7 percentage points.

Metric	Default	Full	Change
P50 (ms)	8.5	8.1	-4.7%
P95 (ms)	20.0	17.2	-14.0%
P99 (ms)	35.0	30.2	-13.7%
Throughput (req/s)	10,000	10,280	+2.8%
Anomaly (%)	3.2	1.5	-53.1%

Table 4: Author-reported measured method comparison. Reduction is relative to the reported 20.0 ms Default value. Run-level uncertainty and matched-budget metadata were not supplied.

Method	P95 (ms)	Reduction
Default	20.0	—
Rules + Graph	19.0	5.0%
BO + Graph	18.2	9.0%
RL + Graph	18.0	10.0%
LLM-only	18.5	7.5%
Graph + LLM	17.4	13.0%
Full SemantOS	17.2	14.0%

Author-Reported Hardware Results

The authors supplied the following aggregate report and subsequently confirmed that its values are actually measured results, correcting its initial “reference result” label. We preserve that provenance distinction: these are author-reported measurements, separate from the locally executed and raw-log-verified experiments in the main text. Run-level CSV files, raw hardware logs, hashes and configuration manifests were not included with the report. Consequently, we can check arithmetic but cannot independently recompute these aggregates, verify the execution claims or derive statistical uncertainty. The report is included without upgrading these limitations into verified evidence.

Reported configuration and execution. The report specifies Linux, eBPF plus userspace telemetry sampled every 5 s, Llama-3.1-8B-Instruct, four workload categories (CPU, memory, I/O, mixed), seven methods and 20 independent runs per workload/method. A complete matrix would contain 560 runs per server; the corresponding run inventory was not supplied. The authors report separate training, validation, calibration and test families, excluding test traces and graph edges from training and retrieval. Exact hardware, kernel release, checkpoint digest, code commit, load settings, budgets and split manifests are needed to make this reported setup reproducible. The supplied description does not identify the executable version used for deployed actuation; the artifact in this revision remains dry-run only.

Method comparisons. The reported Rules + Graph to Graph + LLM reduction is $(19.0 - 17.4)/19.0 = 8.4\%$, and the LLM-only to Graph + LLM reduction is $(18.5 -$

Table 5: Author-reported measured workload P95 values (ms). The supplied Graph label has not been mapped to a specific implementation. These values do not reproduce the aggregate table by unweighted averaging.

Workload	Default	Graph	LLM	G+L	Full
CPU	14.2	13.9	13.7	13.2	13.1
Memory	25.4	23.6	24.0	21.9	21.8
I/O	31.8	29.9	30.7	28.6	28.5
Mixed	22.1	21.2	20.8	19.3	19.1

Table 6: Author-reported measured safety percentages. Counts, denominators and independent unsafe labels were not supplied; these rates alone cannot establish veto recall or rollback precision.

Method	Unsafe (%)	Veto (%)	Rollback (%)
Rules + Graph	8.0	5.1	1.4
BO + Graph	12.5	8.8	2.0
RL + Graph	11.0	7.3	1.9
LLM-only	10.0	6.7	1.8
Graph + LLM	5.5	3.8	1.0
Full SemantOS	4.0	3.1	0.6

17.4)/18.5 = 5.9%. Against the reported BO + Graph and RL + Graph values, Graph + LLM is respectively 4.4% and 3.3% lower. These aggregate contrasts are consistent with a benefit from the combined method but do not by themselves isolate a causal LLM reasoning effect: matched information, budgets, policy differences and paired run-level uncertainty remain to be documented. The full-system 14% difference must not be attributed entirely to the LLM.

Unresolved aggregation. The workload table implies full-system reductions of 7.7%, 14.2%, 10.4% and 13.6%. Its unweighted means are 23.375 ms (Default) and 20.625 ms (Full), rather than the aggregate table’s 20.0 and 17.2 ms. The ratio of those means gives 11.8%, while averaging workload-level reductions gives 11.5%. Neither reproduces 14.0%. An alternative weighting or request-level aggregation may explain the difference, but no rule was supplied; averaging P95 values is not equivalent to a pooled request P95. We therefore retain the two reported tables separately and do not synthesize one headline estimate.

Reported control-loop trace. The supplied trace reports initial P95/P99 of 18.9/31.4 ms, throughput 11,240 req/s and CPU utilization 73.1%. It records recommendations of swapiness 60 $\rightarrow$ 40 and dirty ratio 20 $\rightarrow$ 15 with uncertainty 0.21, passing validation, followed by canary/ramp/full P95 values of 17.4/16.9/16.8 ms and final P99 27.6 ms without rollback. The report also describes range rejection of swapiness -20, and rollback when canary P95 rises from 18.2 to 25.4 ms against a 22.0 ms SLO, followed by P95 18.3 ms. These are author-reported execution excerpts; raw logs, setting readbacks and implementation commit are needed to verify the deployed path and restoration. They do not establish that the current dry-run artifact performs these writes.

Veto precedes deployment; rollback follows an applied change. Their counts and denominators must remain distinct, with independently labeled unsafe actions and unknown labels retained. Nonzero veto or rollback rates alone do not prove effective protection. The reported reduction in unsafe proposals between Graph + LLM and Full SemantOS also requires a description of changes to candidate generation or feedback, because enforcement alone need not change proposed actions.

Reported uncertainty and repetition summary. The report associates scores 0.12 and 0.28 with acceptance, 0.57 with canary/delay, and 0.81 with veto for progressively less familiar workloads. The frozen threshold, calibration sample and implemented delay state were not supplied, so this ordering is not evidence of calibration. Its separate repetition summary gives 20.1 ± 0.9 versus 17.4 ± 0.7 ms, differing from the aggregate table, without defining $\pm$. Six run values are listed (17.12, 17.48, 16.91, 17.35, 17.61, 17.09); twenty complete run records are not present. No confidence interval or significance test is reconstructed from these partial summaries. Identical rounded observations alone would not invalidate measurements, and variation alone would not verify them.

Evidence integration. The report specifies provenance fields for run ID, UTC timestamp, workload family, server ID, kernel release, code commit, method, repetition, split, latency quantiles, throughput, anomaly/window counts, proposal/veto/rollback counts and raw-log SHA-256. These fields describe the expected records; actual records and hashes were not supplied. Retaining proposal acceptance counts, independent anomaly definitions and exposure windows is also necessary. Once available, raw logs should produce run records, aggregates and paper tables through a reproducible pipeline. Until then, the reported reductions are descriptive, author-supplied aggregates with unresolved aggregation and uncertainty, rather than independently reproduced comparative estimates. Chat citation placeholders in the supplied report are not resolvable bibliographic references and are not treated as sources.